ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №7
По дисциплине «Логика и основы
алгоритмизации в инженерных задачах»
На тему «Обход графа в глубину»

Выполнили: ст. гр. 25ВВВ1
Разживалов Н.А.
Цеплов Е.В.
Проверили: к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Обход графа в глубину

Цель работы

Реализовать функцию обхода в глубину для неориентированного графа

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа *G*. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.

3. Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

Ход работы

В теле main реализовали бесконечный цикл while, вовнутрь которого поместили конструкцию switch для выбора пользователем нужного действия. Создали переменные для матрицы смежности и количества вершин.



```
int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));
    int** g = NULL;
    int n;
    int choice;
    Node** gl = NULL;
    while (1) {
        printf("\nвыберите действие и введите его номер:\n 1 - создать граф\n 2 - задать случайные связи вершин графа\n 3 - вы
        scanf_s("%d", &choice);
        switch (choice) {
        case 1: g = createGraph(&n); break;
        case 2: randGraph(g, n); break;
        case 3: printGraph(g, n); break;
        case 4:{
            if (g == NULL) {
                printf("Граф ещё не создан\n");
                break;
            }
            bool* NUM = (bool*)malloc(n * sizeof(bool));
            for (int i = 0; i < n; i++) {
                NUM[i] = false;
            };
            for (int i = 0; i < n; i++)
            {
                if (NUM[i] == false)
                {
                    DFS(i, g, NUM, n);
                }
            }
        }
        break;
        case 5: {
            if (g == NULL) {
                printf("Граф ещё не создан\n");
                break;
            }
            gl = createGraphList(g, n);

            printGraphList(gl, n);

            bool* NUM = (bool*)malloc(n * sizeof(bool));

            for (int i = 0; i < n; i++) {
                NUM[i] = false;
            }

            printf("\nПорядок обхода DFS по списку смежности: ");

            for (int i = 0; i < n; i++) {
                if (NUM[i] == false) {
                    DFSList(i, gl, NUM);
                }
            }

            printf("\n");

            free(NUM);

            freeGraphList(gl, n);
            gl = NULL;
            break;
        }
        case 6: freeGraph(g, n); return 0;
        default: break;
        }
    }
}
```

Основные функции для работы с графом взяли из прошлой лабораторной работы.



```
void freeGraph(int** g, int n) {
    for (int i = 0; i < n; i++) {
        free(g[i]);
    }
    free(g);
}
int** createGraph(int* n) {
    int** g;
    int i;
    printf("Введите количество вершин графа ");
    do {
        scanf_s("%d", n);
        if ((*n) <= 0) {
            printf("введите положительное ненулевое число! ");
        }
    } while ((*n) <= 0);

    g = (int**)malloc((*n) * sizeof(int*));

    for (i = 0; i < (*n); i++) {
        g[i] = (int*)malloc((*n) * sizeof(int));
    }

    return g;
}
```

```
void printGraph(int** g, int n) {
    int i, j;

    if (g == NULL) {
        printf("Граф ещё не был создан\n");
        return;
    }

    printf("\nГраф: \n");

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%d ", g[i][j]);
        }

        printf("\n");
    }
}

void randGraph(int** g, int n) {
    if (g == NULL) {
        printf("Граф ещё не был создан\n");
        return;
    }

    for (int i = 0; i < n; i++) {
        g[i][i] = 0;
    }
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            g[i][j] = rand() % 2;
            g[j][i] = g[i][j];
        }
    }
    printGraph(g, n);
}
```

Реализовали функцию обхода в глубину. Для этого в main заполнили дополнительный логический массив NUM значениями false. Далее происходит первый вызов функции. Для первого элемента присваивается значение true, а затем находится следующая вершина, которая ещё не была отмечена, и происходит рекурсивный вызов. В момент, когда последующих вершин нет, происходит возврат к тому вызову, где ещё остались не посещённые вершины.

```
case 4:{
    if (g == NULL) {
        printf("Граф ещё не создан\n");
        break;
    }
    bool* NUM = (bool*)malloc(n * sizeof(bool));
    for (int i = 0; i < n; i++) {
        NUM[i] = false;
    };
    for (int i = 0; i < n; i++)
    {
        if (NUM[i] == false)
        {
            DFS(i, g, NUM, n);
        }
    }
}
```

```
void DFS(int v, int** g, bool* NUM, int n)
{
    NUM[v] = true;

    printf("%d ", v + 1);

    for (int i = 0; i < n; i++) {
        if (g[v][i] == 1 && NUM[i] == false) {
            DFS(i, g, NUM, n);
        }
    }
}
```

Создали структуру для реализации списка смежности.

```
struct Node {
    int vertex;
    Node* next;
};
```

Реализовали заполнение списка смежности на основе исходной матрицы смежности. Реализовали функции вывода и освобождения памяти для списка смежности.

```
Node** createGraphList(int** g, int n)
{
    Node** gl = (Node**)malloc(n * sizeof(Node*));

    for (int i = 0; i < n; i++) {
        gl[i] = NULL;
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

            if (g[i][j] == 1) {

                Node* newNode = (Node*)malloc(sizeof(Node));

                newNode->vertex = j;
                newNode->next = NULL;
                if (gl[i] == NULL) {
                    gl[i] = newNode;
                }
                else {
                    Node* current = gl[i];

                    while (current->next != NULL) {
                        current = current->next;
                    }

                    current->next = newNode;
                }
            }
        }
    }

    return gl;
}
```

```
void printGraphList(Node** gl, int n)
{
    printf("\nСписок смежности:\n");

    for (int i = 0; i < n; i++) {
        printf("%d: ", i + 1);

        Node* current = gl[i];

        while (current != NULL) {
            printf("%d ", current->vertex + 1);
            current = current->next;
        }

        printf("\n");
    }
}

void freeGraphList(Node** gl, int n)
{
    for (int i = 0; i < n; i++) {

        Node* current = gl[i];

        while (current != NULL) {
            Node* temp = current;
            current = current->next;
            free(temp);
        }
    }

    free(gl);
}
```

Реализовали функцию обхода в глубину для списка смежности. Функция работает по аналогичному алгоритму, только проверяет сразу инцидентные элементы для каждой вершины

```
void DFSList(int v, Node** gl, bool* NUM)
{
    NUM[v] = true;

    printf("%d ", v + 1);

    Node* current = gl[v];

    while (current != NULL) {

        int nextVertex = current->vertex;

        if (NUM[nextVertex] == false) {
            DFSList(nextVertex, gl, NUM);
        }

        current = current->next;
    }
}
```

Для реализации нерекурсивного обхода взяли структуру стека и функции для работы с ним из одной из прошлых работ и преобразовали её для работы с числами (номерами вершин).

```
struct stack
{
    int vertex;
    struct stack* next;
};

struct stack* head = NULL;


struct Node {
    int vertex;
    Node* next;
};

void push(struct stack** head, int v) {
    struct stack* p = (struct stack*)malloc(sizeof(struct stack));
    if (p == NULL) {
        printf("Ошибка выделения памяти\n");
        return;
    }
    p->vertex = v;
    p->next = *head;
    *head = p;
}

int pop(struct stack** head) {
    if (*head == NULL) {
        return -1;
    }
    struct stack* p = *head;
    int v = p->vertex;
    *head = (*head)->next;
    free(p);
    return v;
}

void clear(struct stack** head) {
    struct stack* p;
    while (*head != NULL) {
        p = *head;
        *head = (*head)->next;
        free(p);
    }
}
```

Реализовали нерекурсивный обход на основе исходных функций, преобразовав их под работу со стеком.

```
void DFS(int start, int** g, bool* NUM, int n) {
    struct stack* head = NULL;

    push(&head, start);

    while (head != NULL) {
        int v = pop(&head);

        if (NUM[v] == true) {
            continue;
        }

        NUM[v] = true;
        printf("%d ", v + 1);

        for (int i = n - 1; i >= 0; i--) {
            if (g[v][i] == 1 && NUM[i] == false) {
                push(&head, i);
            }
        }
    }

    clear(&head);
}
```

```
Node** createGraphList(int** g, int n)
{
    Node** gl = (Node**)malloc(n * sizeof(Node*));

    for (int i = 0; i < n; i++) {
        gl[i] = NULL;
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

            if (g[i][j] == 1) {

                Node* newNode = (Node*)malloc(sizeof(Node));

                newNode->vertex = j;
                newNode->next = NULL;
                if (gl[i] == NULL) {
                    gl[i] = newNode;
                }
                else {
                    Node* current = gl[i];

                    while (current->next != NULL) {
                        current = current->next;
                    }

                    current->next = newNode;
                }
            }
        }
    }

    return gl;
}
```

Псевдокод

Рекурсивный обход

```
АЛГОРИТМ DFS(v, g, NUM, n)
ВХОД:
   v   : ЦЕЛОЕ
   g   : МАССИВ [n, n]
   NUM : МАССИВ [n]
   n   : ЦЕЛОЕ
ВЫХОД:
   Вывод порядка обхода на экран
НАЧАЛО
   NUM[v] := ИСТИНА
   ВЫВЕСТИ(v + 1)
   ДЛЯ i ОТ 0 ДО n - 1 ЦИКЛ
      ЕСЛИ g[v][i] = 1 И NUM[i] = ЛОЖЬ ТОГДА
         DFS(i, g, NUM, n)
      КОНЕЦ_ЕСЛИ
   КЦ
КОНЕЦ
```

Нерекурсивный обход

```
АЛГОРИТМ DFS (start, g, NUM, n)
ВХОД:
   start : ЦЕЛОЕ
   g     : МАССИВ [n, n]
   NUM   : МАССИВ [n]
   n     : ЦЕЛОЕ
ВЫХОД:
   Вывод порядка обхода на экран
НАЧАЛО
   head := 0
   PUSH(&head, start)
   ПОКА head ≠ 0 ЦИКЛ
      v := POP(&head)
      ЕСЛИ NUM[v] = ИСТИНА ТОГДА
         ПРОДОЛЖИТЬ_ЦИКЛ
      КОНЕЦ_ЕСЛИ
      NUM[v] := ИСТИНА
```

```
      ВЫВЕСТИ(v + 1)
      ДЛЯ i ОТ n - 1 ДО 0 С ШАГОМ -1 ЦИКЛ
        ЕСЛИ g[v][i] = 1 И NUM[i] = ЛОЖЬ ТОГДА
          PUSH(&head, i)
        КОНЕЦ_ЕСЛИ
      КЦ
    КЦ
    CLEAR(&head)
КОНЕЦ
```

Рекурсивный обход списка смежности

```
АЛГОРИТМ DFSList(v, gl, NUM)
ВХОД:
    v   : ЦЕЛОЕ
    gl  : МАССИВ УКАЗАТЕЛЕЙ
    NUM : МАССИВ [n]
ВЫХОД:
    Вывод порядка обхода на экран
НАЧАЛО
    NUM[v] := ИСТИНА
    ВЫВЕСТИ(v + 1)
    current := gl[v]
    ПОКА current ≠ 0 ЦИКЛ
      nextVertex := current->vertex
      ЕСЛИ NUM[nextVertex] = ЛОЖЬ ТОГДА
        DFSList(nextVertex, gl, NUM)
      КОНЕЦ_ЕСЛИ
      current := current->next
    КЦ
КОНЕЦ
```

Нерекурсивный обход списка смежности

```
АЛГОРИТМ DFSList (start, gl, NUM)
ВХОД:
    start : ЦЕЛОЕ
    gl    : МАССИВ УКАЗАТЕЛЕЙ
    NUM   : МАССИВ [n]
ВЫХОД:
    Вывод порядка обхода на экран
```

```
НАЧАЛО
   head := 0
   PUSH(&head, start)
   ПОКА head ≠ 0 ЦИКЛ
      v := POP(&head)
      ЕСЛИ NUM[v] = ИСТИНА ТОГДА
         ПРОДОЛЖИТЬ_ЦИКЛ
      КОНЕЦ_ЕСЛИ
      NUM[v] := ИСТИНА
      ВЫВЕСТИ(v + 1)
      current := gl[v]
      ПОКА current ≠ НУЛЬ ЦИКЛ
         nextVertex := current->vertex
         ЕСЛИ NUM[nextVertex] = ЛОЖЬ ТОГДА
            PUSH(&head, nextVertex)
         КОНЕЦ_ЕСЛИ
         current := current->next
      КЦ
   КЦ
   CLEAR(&head)
КОНЕЦ
```

Результаты работы программы

```
Введите количество вершин графа 7

выберите действие и введите его номер:
 1 - создать граф
 2 - задать случайные связи вершин графа
 3 - вывести граф
 4 - осуществить обход в глубину
 5 - осуществить обход в глубину через списки смежности
2

Граф:
0 1 0 0 1 1 0
1 0 0 0 1 0 1
0 0 0 0 0 0 1
0 0 0 0 1 1 1
1 1 0 1 0 1 0
1 0 0 1 1 0 0
0 1 1 1 0 0 0

выберите действие и введите его номер:
 1 - создать граф
 2 - задать случайные связи вершин графа
 3 - вывести граф
 4 - осуществить обход в глубину
 5 - осуществить обход в глубину через списки смежности
4
1 2 5 4 6 7 3
5

Список смежности:
1: 2 5 6
2: 1 5 7
3: 7
4: 5 6 7
5: 1 2 4 6
6: 1 4 5
7: 2 3 4

Порядок обхода DFS по списку смежности: 1 2 5 4 6 7 3
```

```
Граф:
0 0 0 1 1 0 0
0 0 1 0 0 1 0
0 1 0 1 0 1 1
1 0 1 0 1 0 1
1 0 0 1 0 0 1
0 1 1 0 0 0 0
0 0 1 1 1 0 0

выберите действие и введите его номер:
 1 - создать граф
 2 - задать случайные связи вершин графа
 3 - вывести граф
 4 - осуществить обход в глубину
 5 - осуществить обход в глубину через списки смежности
4

Порядок нерекурсивного обхода: 1 4 3 2 6 7 5
5

Список смежности:
1: 4 5
2: 3 6
3: 2 4 6 7
4: 1 3 5 7
5: 1 4 7
6: 2 3
7: 3 4 5

Порядок обхода по списку смежности: 1 4 3 2 6 7 5
```

Вывод

Реализовали функцию обхода в глубину для неориентированного графа